



Embedded Systems Developer Intern & New Graduate Candidate Assignment

12.04.2026

Hamza AKSİN

Electrical & Electronics Engineering / Engineering Faculty - Marmara University

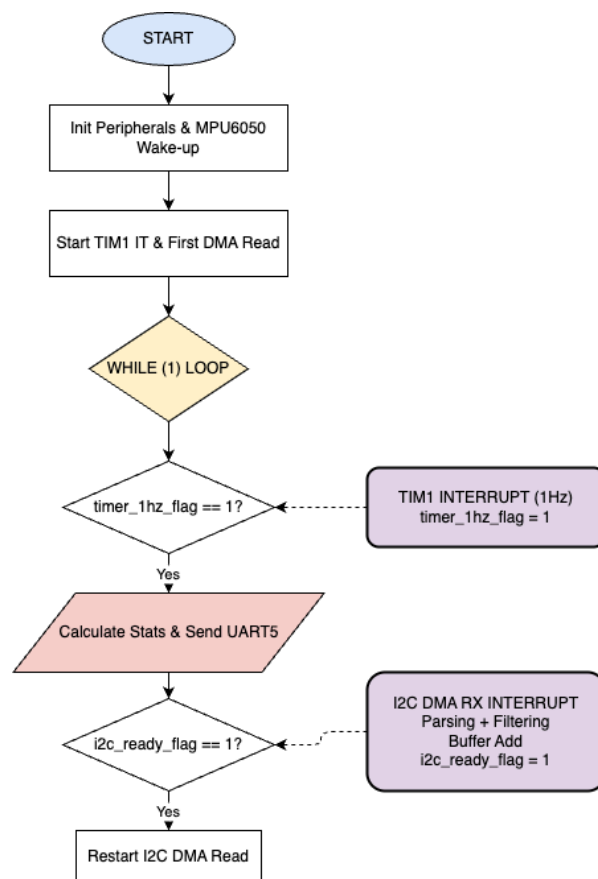
Maltepe, Istanbul

1. Project Overview.....	2
2. System Architecture & Flowchart.....	2
3. I2C Communication & Data Parsing (I2C Haberleşme ve Veri İşleme).....	2
4. Signal Processing: Moving Median Filter (Sinyal İşleme: Medyan Filtre).....	3
5. Data Management: Circular Buffer (Veri Yönetimi: Dairesel Tampon).....	3
6. Periodic Reporting & Statistics (Periyodik Raporlama ve İstatistik).....	4
7. I'd like to add some.....	5

1. Project Overview

I used the STM32 platform for this project because I have been conducting my own studies on it. Currently, the only sensor I have available is the MPU6050. Although there is a file named MPU9250 in the project, I did not use it since I2C communication was required, and it is largely the same as the MPU6050 for this purpose. My microcontroller is the STM32F407VGT6. As it does not feature a built-in BLE function, I implemented a USB-TTL converter to transmit the filtered data at a 1 Hz frequency.

2. System Architecture & Flowchart



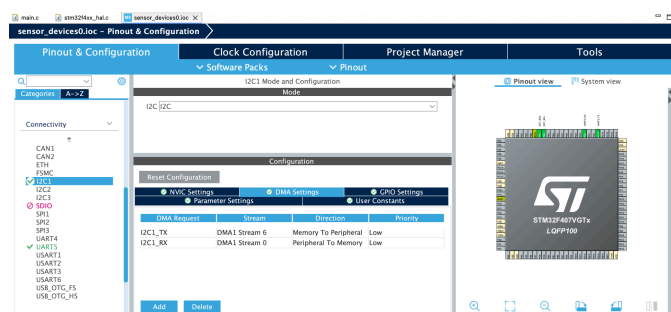
In this project, I utilized DMA, TIM Interrupt, and UART5 for specific operational tasks. First, the peripherals and the sensor are initialized. Then, the timer interrupt is started to ensure that the data is transmitted at a consistent 1 Hz frequency. Simultaneously, the DMA reading process is initiated to handle data transfer without CPU overhead.

The code then enters the while(1) loop. It first checks whether the timer flag is set. If the flag is active, the system calculates the required statistics—standard deviation, maximum, minimum, and median—and transmits the results via UART5. After this process, the loop checks the sensor ready flag (I2C ready flag) to ensure the data acquisition cycle is complete before starting the next iteration.

3. I2C Communication & Data Parsing

In this project, efficiency was a priority. To handle sensor readings without overloading the CPU, I implemented Direct Memory Access (DMA).

Instead of making the processor wait for each byte from the MPU6050, I



configured the I2C1 peripheral to use DMA. This allows the CPU to trigger the read command and immediately return to other tasks while data is transferred to RAM in the background. Once the 14-byte packet is fully received, an interrupt is triggered to process the data.

The raw data from the sensor consists of two 8-bit registers (High and Low) for each axis. In my code, I reconstruct these into 16-bit integers by shifting and combining the bytes: $(\text{High} \ll 8) | \text{Low}$.

After reconstruction, I convert these raw values into meaningful physical units (float) such as g for acceleration and deg/s for angular velocity, based on the sensitivity scales defined in the sensor's datasheet.

4. Signal Processing: Moving Median Filter

I implemented a Moving Median Filter because it was a specific requirement of the assignment to ensure clean and reliable sensor data. The filter is designed to eliminate "spiky" noise and outliers that are common in IMU sensors like the MPU6050.

I developed the filtering logic in dedicated files, `median_filter.c` and `median_filter.h`, to keep the project modular and organized.

The filter is called within the `HAL_I2C_MemRxCpltCallback` in `main.c`. Every time a new raw value is parsed, it is immediately passed through the filter before being stored.

Following the project guidelines, the filter uses a specific window size (number of samples) to perform its calculations. It stores the most recent samples in a small array, sorts them, and selects the middle value (the median) as the output. This approach effectively ignores extreme noise spikes while maintaining the true trend of the environmental data.

5. Data Management: Circular Buffer

To manage the filtered sensor data efficiently, I implemented a Circular Buffer (Ring Buffer). This structure is essential for keeping a continuous window of samples for statistical analysis while maintaining a fixed memory footprint.

For optimal memory management, I used statically allocated buffers instead of dynamic allocation. This prevents heap fragmentation and ensures that memory usage is determined at compile-time, which is a safer and more predictable approach for embedded systems like the STM32.

The buffer operates on a "First-In, First-Out" (FIFO) logic using head and tail indices. When the buffer reaches its maximum capacity (overflow), the system is designed to

The core logic is encapsulated in `circular_buffer.c` and `circular_buffer.h`. These files contain the dedicated data structures and functions, such as `buffer_add_value`, which manage index wrapping and ensure data

buf_ay	buf_handle {...}	buf_ax	buf_handle {...}
buffer	float [100] [100]	buffer	float [100] [100]
0	-0.00756835938	0	-0.00708007812
1	-0.0068359375	1	-0.00659179688
2	-0.0068359375	2	-0.0078125
3	-0.0068359375	3	-0.0078125
4	-0.0068359375	4	-0.00952148438
5	-0.0068359375	5	-0.00952148438
6	-0.00756835938	6	-0.00952148438
7	-0.00756835938	7	-0.00952148438
8	-0.00756835938	8	-0.00952148438
9	-0.00756835938	9	-0.00659179688
10	-0.00756835938	10	-0.00415039062
11	-0.00952148438	11	-0.00415039062
12	-0.00952148438	12	-0.0048828125
13	-0.00952148438	13	-0.0068359375
14	-0.00952148438	14	-0.00854492188
15	-0.00659179688	15	-0.00854492188
16	-0.00634765625	16	-0.0107421875
17	-0.00659179688	17	-0.0107421875
18	-0.00659179688	18	-0.00927734375
19	-0.00634765625	19	-0.00927734375
20	-0.00830078125	20	-0.01171875
21	-0.00830078125	21	-0.00927734375
22	-0.00805664062	22	-0.00927734375
23	-0.00610351562	23	-0.00830078125
24	-0.00805664062	24	-0.0078125
25	-0.0078125	25	-0.0078125
26	-0.0078125	26	-0.0078125
27	-0.00708007812	27	-0.0078125
28	-0.00708007812	28	-0.00732421875
29	-0.00708007812	29	-0.00732421875
30	-0.00708007812	30	-0.00732421875
31	-0.00708007812	31	-0.00659179688
32	-0.00830078125	32	-0.00659179688
33	-0.00830078125	33	-0.00732421875

6. Periodic Reporting & Statistics

As you can see from the images, the reporting phase of the project is managed by the TIM1 Interrupt. This hardware timer is configured to trigger every second to maintain a precise 1 Hz measurement frequency. At each interval, the system processes the values stored in the

circular buffer to calculate essential statistics: Maximum, Minimum, Median, and Standard Deviation.

Regarding the transmission, I utilized UART5 instead of Bluetooth Low Energy (BLE). Since the STM32F407VGT6 does not feature a native BLE peripheral, I implemented a USB-TTL converter to output the data. I chose this method for rapid prototyping and verification purposes, as it allowed me to monitor the filtered data and statistical reports in real-time through the Arduino Serial Monitor.

```

sketch_apr12a | Arduino IDE 2.3.7
ALKS ESP32
sketch_apr12a.ino
1 void setup() {
Serial Monitor x
Message (Enter to send message to 'ALKS ESP32' on '/dev/cu.usbserial-1120')
[ACC Y] Max:-0.00 Min:-0.02 Med:-0.01 StdDev:0.0027
[ACC Z] Max:1.01 Min:1.00 Med:1.01 StdDev:0.0038
[GYRO X] Max:-0.45 Min:-0.87 Med:-0.64 StdDev:0.0923
[GYRO Y] Max:-0.86 Min:-1.78 Med:-1.31 StdDev:0.1665
[GYRO Z] Max:-0.27 Min:-0.74 Med:-0.49 StdDev:0.1094
--- [END OF REPORT] ---
--- [1Hz SENSOR REPORT] ---
[ACC X] Max:-0.01 Min:-0.02 Med:-0.02 StdDev:0.0024
[ACC Y] Max:-0.00 Min:-0.01 Med:-0.01 StdDev:0.0019
[ACC Z] Max:1.01 Min:1.00 Med:1.01 StdDev:0.0043
[GYRO X] Max:-0.45 Min:-0.85 Med:-0.61 StdDev:0.0928
[GYRO Y] Max:-0.86 Min:-1.78 Med:-1.27 StdDev:0.1695
[GYRO Z] Max:-0.26 Min:-0.74 Med:-0.47 StdDev:0.1041
--- [END OF REPORT] ---
--- [1Hz SENSOR REPORT] ---
[ACC X] Max:-0.02 Min:-0.02 Med:-0.02 StdDev:0.0023
[ACC Y] Max:-0.00 Min:-0.01 Med:-0.01 StdDev:0.0011
[ACC Z] Max:1.01 Min:1.00 Med:1.01 StdDev:0.0032
[GYRO X] Max:-0.44 Min:-0.77 Med:-0.63 StdDev:0.0781
[GYRO Y] Max:-1.07 Min:-1.58 Med:-1.31 StdDev:0.1293
[GYRO Z] Max:-0.26 Min:-0.76 Med:-0.50 StdDev:0.1075
--- [END OF REPORT] ---
--- [1Hz SENSOR REPORT] ---
[ACC X] Max:-0.02 Min:-0.02 Med:-0.02 StdDev:0.0018
[ACC Y] Max:-0.01 Min:-0.01 Med:-0.01 StdDev:0.0009
[ACC Z] Max:1.01 Min:1.00 Med:1.00 StdDev:0.0032
[GYRO X] Max:-0.47 Min:-0.77 Med:-0.63 StdDev:0.0748
[GYRO Y] Max:-1.02 Min:-1.58 Med:-1.32 StdDev:0.1122
[GYRO Z] Max:-0.24 Min:-0.76 Med:-0.50 StdDev:0.1149
--- [END OF REPORT] ---

```

7. I'd like to add some...

I applied for this internship program on 30 March, and my midterm exams started on 6 April. Therefore, I couldn't find enough time to prepare all the tasks. Today is 12 April, and I've started over and am finishing it on the same day. With all the errors, bugs, and the preparation of the report, I would like one more chance, such as an interview, to present a more well-structured project because I don't think I could show my abilities in this one. Thanks for your understanding.

8. Links

1. <https://cdn.sparkfun.com/datasheets/Sensors/Accelerometers/RM-MPU-6000A.pdf>
2. https://github.com/hamzaaksin/sensor_devices.git (Use master branch.)